МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторной работе № 5

Тема: «Реализация алгоритмов поиска аномалий в данных: итерация по временному ряду, выявление выбросов по пороговым и статистическим критериям и формирование очищенного массива»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория

Параметр	Значение
Дисциплина	«Программирование и прототипирование ИТ-решений»
Направление подготовки	09.03.03 Прикладная информатика
Профиль	Бизнес и системная аналитика
Семестр / трудоёмкость	1 семестр, 4 академических часа (2 пары)
Среда разработки	Visual Studio Code
Форма контроля	отчёт о выполнении работы + устная защита
Обеспеченность ПО	Python 3.11+, Git, VS Code (подготовлены в ЛР № 1)

1. Цель работы

Научиться программно итерироваться по массивам (спискам) технологических измерений, выявлять аномальные значения (выбросы) с помощью жёстких технологических границ и статистических критериев (метод z-отклонений / k·σ и метод межквартильного размаха IQR) без использования сторонних библиотек, а также формировать очищенный (отфильтрованный) массив и ряд с замещением аномалий на примере часовых замеров забойного давления нефтяной скважины.

В результате выполнения работы студент научится:

- итерироваться по временному ряду с одновременным контролем индекса (номера часа) и значения измерения через enumerate();
- реализовывать собственным кодом расчёт выборочного стандартного отклонения (σ) и квартилей ряда (Q1, Q3, межквартильный размах IQR);
- разрабатывать алгоритмы детекции выбросов на основе технологического коридора [P_min; P_max], статистического правила k·σ (z-score) и робастного метода Тьюки (1.5·IQR);
- формировать очищенный массив данных (фильтрация выбросов) и массив с восстановлением пропусков (замещение аномалий медианой);
- сравнивать статистические характеристики ряда ДО и ПОСЛЕ очистки, сверять программный расчёт с ручным контрольным пересчётом и фиксировать этапы работы в Git.

2. Обязательное ядро (одинаково для обоих вариантов)

- скрипт формирует массив индивидуальных замеров забойного давления по номеру студента N, содержащий как нормальные измерения, так и искусственные сбои датчика («шпильки» и провалы);

- все расчёты статистических порогов (среднее, стандартное отклонение, медиана, квартили Q1/Q3, границы отсечения) выполняются собственным алгоритмом на чистом Python (без импорта statistics, numpy, pandas, scipy);
- реализована проверка каждого измерения на вхождение в допустимый диапазон и классификация точек на «Норма» и «Аномалия (выброс)» с сохранением индексов (часов) сбоев;
- формируется новый отфильтрованный список очищенных данных без разрушения исходного списка замеров;
- выводится сравнительная таблица статистик ряда (среднее, медиана, СКО, размах) ДО очистки и ПОСЛЕ очистки с точностью до двух знаков после запятой (:.2f);
- в репозитории Git сделано не менее трёх осмысленных коммитов (инициализация → алгоритмы поиска и фильтрации аномалий → документация);
- в отчёт включён ручной контрольный пересчёт стандартного отклонения (или квартилей Q1, Q3 и границ IQR), порогов отсечения и списка найденных выбросов для ВАШИХ индивидуальных данных;
- выполнены ВСЕ 7 обязательных упражнений на понимание (раздел 8) с фиксацией результатов и скриншотов в отчёте;
- README.md содержит описание алгоритма детекции аномалий, математические формулы порогов, пример запуска и пример табличного вывода.

3. Глоссарий: все термины работы

Перед началом работы внимательно изучите термины. Они понадобятся при написании кода и на устной защите.

3.1. Аномалии, выбросы и методы их фильтрации

Термин	Что это значит
Аномалия (Выброс / Outlier)	Измерение, которое резко отклоняется от общего уровня ряда и физической нормы процесса (например, скачок давления с 180 до 265 атм из-за сбоя телеметрии или гидроудара).
Технологические границы [P_min; P_max]	Жёсткие физические или регламентные пределы показателя, заданные технологом (например, забойное давление не может быть отрицательным или превышать предел прочности колонны).
Статистический порог	Граница нормы, которая вычисляется автоматически из самого ряда данных (через среднее и стандартное отклонение либо через квартили).
Фильтрация (Удаление выбросов)	Формирование нового массива, в который переносятся только нормальные замеры, а все аномальные точки исключаются (длина списка уменьшается).
Импутация (Замещение / Imputation)	Замена обнаруженного аномального значения на расчётную оценку (например, на медиану ряда или среднее соседних точек) для сохранения длины временного ряда.

3.2. Статистические критерии поиска выбросов (k·σ и IQR)

Термин	Что это значит

Дисперсия (Variance, D)	Средний квадрат отклонений элементов ряда от их среднего арифметического: D = sum((x - mean)**2) / n.
Стандартное отклонение (СКО, σ / std)	Квадратный корень из дисперсии: std = D ** 0.5. Показывает типичный разброс значений вокруг среднего в тех же единицах измерения (атм).
Z-оценка (Z-score)	Безразмерная величина z = abs(x - mean) / std, показывающая, на сколько стандартных отклонений точка x удалена от среднего.
Правило k·σ (метод Z-score)	Точка считается выбросом, если она вышла за интервал [mean - k*std; mean + k*std] (то есть z > k). Для коротких рядов с одиночными сбоями обычно берут k = 2.0.
Квартили Q1 и Q3	Значения, делящие отсортированный ряд на четверти: Q1 (25-й перцентиль) — медиана младшей половины ряда; Q3 (75-й перцентиль) — медиана старшей половины ряда.
Межквартильный размах (IQR)	Разность между третьим и первым квартилями: IQR = Q3 - Q1. Описывает ширину центральной («надёжной») половины данных и не зависит от экстремальных выбросов.
Метод Тьюки (1.5·IQR)	Робастный критерий: выбросами признаются все точки ниже нижней изгороди Low = Q1 - 1.5*IQR или выше верхней изгороди High = Q3 + 1.5*IQR.

3.3. Конструкции Python для итерации и фильтрации

Термин	Что это значит
enumerate(series, start=1)	Позволяет в цикле for получать одновременно порядковый номер замера (час) и само значение: for hour, val in enumerate(series, start=1):.
Фильтрующее включение	Конструкция [x for x in series if low <= x <= high], создающая новый очищенный список в одну строку без изменения исходного списка.
Тернарный оператор замены	Выражение x if low <= x <= high else median_val, позволяющее заменить выброс на медиану прямо при обходе списка.
abs(x)	Встроенная функция вычисления модуля (абсолютной величины) числа: abs(-15.4) вернёт 15.4.
x ** 0.5	Возведение в степень 0.5 — вычисление квадратного корня числа без импорта модуля math.

4. Необходимое программное обеспечение

Используется стандартное окружение, настроенное в лабораторной работе № 1:

Инструмент	Назначение	Проверка
Python 3.11+	интерпретатор языка	python --version
Visual Studio Code	редактор кода и терминал	окно VS Code открывается

Git	система контроля версий	git --version
Виртуальное окружение	изоляция проекта	(.venv) в начале строки терминала

5. Общая подготовка (после ЛР № 1–4)

Шаг 5.1. Проверить версии инструментов

```
python --version
git --version
```

Зачем это делаем: Убеждаемся, что интерпретатор Python и система Git доступны из терминала VS Code.

Шаг 5.2. Создать каталог проекта

Создайте отдельную папку под лабораторную работу № 5 (имя зависит от выбранного варианта — см. разделы 6 и 7):

```
mkdir anomaly-basic
```

Зачем это делаем: Каждая лабораторная работа размещается в собственном изолированном Git-репозитории.

Шаг 5.3. Создать и активировать виртуальное окружение

```
python -m venv .venv
.\.venv\Scripts\Activate
```

В VS Code выберите интерпретатор созданного окружения: Ctrl + Shift + P → Python: Select Interpreter → .venv.

6. Вариант А (базовый): «Выявление сбоев датчика забойного давления и очистка ряда»

6.1. Что мы делаем

Создаём консольный скрипт, который формирует 12-часовой ряд замеров забойного давления скважины (в атмосферах), вычисляет статистические параметры ряда (среднее, медиану, стандартное отклонение σ) и выполняет двухэтапный поиск аномалий: 1) по жёстким технологическим границам [P_min; P_max]; 2) по статистическому критерию двух сигм (z-score > 2.0). Затем программа формирует очищенный массив и наглядно показывает, как после удаления всего двух сбоев датчика нормализуются среднее арифметическое и размах вариации.

6.2. Индивидуальные исходные данные (Вариант А)

Индивидуальный ряд из 12 часовых замеров забойного давления P (атм) рассчитывается по вашему номеру N в журнале группы. В ряд специально заложены два резких аппаратных сбоя телеметрии: провал на 4-м часу и высоковольтный всплеск («шпилька») на 9-м часу.

Формула генерации индивидуального ряда из 12 замеров (Вариант А):

```
N = ваш номер по журналу (например, N = 5)
P0 = 160.0 + 2.0 * N        # базовое забойное давление, атм (для N = 5: P0 = 170.0)
S  = (N % 4) + 1.0          # шаг естественных колебаний, атм (для N = 5: S = 2.0)

# Коэффициенты отклонений на 12 часов (индексы 0..11, часы 1..12):
# Час 4 (коэф. -18.0) — резкий провал сигнала; Час 9 (коэф. +22.0) — аварийный всплеск!
Коэффициенты K: [0.0, 1.0, -1.0, -18.0, 2.0, 0.0, -2.0, 1.0, 22.0, -1.0, 1.0, 2.0]
Формула каждого часа: P_i = P0 + K_i * S

Технологический регламент скважины (жёсткие физические границы):
P_min = P0 - 10.0 * S       (для N = 5: 170.0 - 20.0 = 150.00 атм)
P_max = P0 + 10.0 * S       (для N = 5: 170.0 + 20.0 = 190.00 атм)
```

Перед запуском программы выпишите на бумаге все 12 значений вашего ряда для своего N. Например, при N = 5 (P0 = 170.0, S = 2.0) получаем ряд: [170.0, 172.0, 168.0, 134.0, 174.0, 170.0, 166.0, 172.0, 214.0, 168.0, 172.0, 174.0].

Шаг 1. Создать проект

```
mkdir anomaly-basic
cd anomaly-basic
python -m venv .venv
.\.venv\Scripts\Activate
```

Создайте в корне проекта файл main.py.

Шаг 2. Написать код

Внимательно разберите каждую функцию и перенесите код в файл main.py:

Полный код файла main.py (Вариант А):

```
"""Выявление аномалий и очистка ряда забойного давления скважины (вариант А)."""

Z_THRESHOLD = 2.0  # пороговый коэффициент k для статистического правила k * sigma


def generate_pressure_series(student_n: int) -> tuple[list[float], float, float]:
    """Формирует 12-часовой ряд давления P (атм) и возвращает (ряд, P_min, P_max)."""
    p_base = 160.0 + 2.0 * student_n
    step = float((student_n % 4) + 1)
    offsets = [0.0, 1.0, -1.0, -18.0, 2.0, 0.0, -2.0, 1.0, 22.0, -1.0, 1.0, 2.0]

    series: list[float] = []
    for k in offsets:
        series.append(round(p_base + k * step, 2))

    p_min = round(p_base - 10.0 * step, 2)
    p_max = round(p_base + 10.0 * step, 2)
```

```
    return series, p_min, p_max


def calc_mean(series: list[float]) -> float:
    """Вычисляет среднее арифметическое числового ряда."""
    if not series:
        raise ValueError("Пустой массив данных")
    total = 0.0
    for val in series:
        total += val
    return total / len(series)


def calc_median(series: list[float]) -> float:
    """Вычисляет медиану ряда без изменения исходного списка."""
    n = len(series)
    if n == 0:
        raise ValueError("Пустой массив данных")
    ordered = sorted(series)
    mid = n // 2
    if n % 2 == 1:
        return ordered[mid]
    return (ordered[mid - 1] + ordered[mid]) / 2.0


def calc_std(series: list[float], mean_val: float) -> float:
    """Вычисляет стандартное отклонение (СКО, sigma) числового ряда."""
    if not series:
        raise ValueError("Пустой массив данных")
    sq_diff_sum = 0.0
    for val in series:
        sq_diff_sum += (val - mean_val) ** 2
    variance = sq_diff_sum / len(series)
    return variance ** 0.5


def calc_range(series: list[float]) -> tuple[float, float, float]:
    """Возвращает (минимум, максимум, размах вариации) за один проход."""
    if not series:
        raise ValueError("Пустой массив данных")
    lo = hi = series[0]
    for val in series[1:]:
        if val < lo:
            lo = val
        if val > hi:
            hi = val
    return lo, hi, round(hi - lo, 2)


def filter_anomalies(
    series: list[float],
    low_bound: float,
    high_bound: float,
) -> tuple[list[float], list[tuple[int, float]]]:
    """Разделяет ряд на очищенный список и список кортежей аномалий (час, значение)."""
    cleaned: list[float] = []
    anomalies: list[tuple[int, float]] = []
    for hour, val in enumerate(series, start=1):
```

```
        if low_bound <= val <= high_bound:
            cleaned.append(val)
        else:
            anomalies.append((hour, val))
    return cleaned, anomalies


def main() -> None:
    print("ПОИСК АНОМАЛИЙ И ОЧИСТКА РЯДА ЗАБОЙНОГО ДАВЛЕНИЯ (ВАРИАНТ А)")
    print("=" * 68)
    raw_n = input("Введите ваш номер по журналу N: ").strip()
    student_n = int(raw_n) if raw_n.isdigit() and int(raw_n) > 0 else 5

    pressures, tech_min, tech_max = generate_pressure_series(student_n)

    # Статистики сырого («грязного») ряда
    raw_mean = calc_mean(pressures)
    raw_med = calc_median(pressures)
    raw_std = calc_std(pressures, raw_mean)
    raw_min, raw_max, raw_rng = calc_range(pressures)

    # Статистические границы по правилу k * sigma
    stat_low = round(raw_mean - Z_THRESHOLD * raw_std, 2)
    stat_high = round(raw_mean + Z_THRESHOLD * raw_std, 2)

    print(f"\nИсходный ряд (12 ч):   {pressures}")
    print(f"Тех. регламент:        [{tech_min:.2f}; {tech_max:.2f}] атм")
    print(f"Границы {Z_THRESHOLD:.1f}*sigma:     [{stat_low:.2f}; {stat_high:.2f}] атм")

    print("\nДЕТАЛЬНАЯ ТАБЛИЦА КОНТРОЛЯ ИЗМЕРЕНИЙ")
    print("-" * 68)
    print(f"{'Час':<5}{'P, атм':>10}{'Откл. от ср.':>14}{'Z-оценка':>11}{'Тех.
коридор':>15}{'Статус точки':>13}")
    print("-" * 68)

    for hour, val in enumerate(pressures, start=1):
        diff = val - raw_mean
        z_score = abs(diff) / raw_std if raw_std > 0 else 0.0
        in_tech = "ОК" if tech_min <= val <= tech_max else "НАРУШЕН"
        is_outlier = (val < stat_low or val > stat_high) or (in_tech == "НАРУШЕН")
        status = "ВЫБРОС!" if is_outlier else "Норма"

print(f"{hour:<5}{val:>10.2f}{diff:>+14.2f}{z_score:>11.2f}{in_tech:>15}{status:>13}")

    # Фильтрация ряда по статистическим границам
    cleaned_series, found_anomalies = filter_anomalies(pressures, stat_low, stat_high)

    # Статистики очищенного ряда
    cln_mean = calc_mean(cleaned_series)
    cln_med = calc_median(cleaned_series)
    cln_std = calc_std(cleaned_series, cln_mean)
    _, _, cln_rng = calc_range(cleaned_series)

    print("\nРЕЗУЛЬТАТ ФИЛЬТРАЦИИ АНОМАЛИЙ")
    print("-" * 68)
    print(f"Обнаружено выбросов:   {len(found_anomalies)} шт. -> {found_anomalies}")
    print(f"Очищенный ряд ({len(cleaned_series)} ч):  {cleaned_series}")
```

```
    print("\nСРАВНЕНИЕ СТАТИСТИК ДО И ПОСЛЕ ОЧИСТКИ")
    print("-" * 68)
    print(f"{'Показатель':<28}{'До очистки':>14}{'После очистки':>16}{'Изменение':>10}")
    print("-" * 68)
    print(f"{'Среднее арифметическое, атм':<28}{raw_mean:>14.2f}{cln_mean:>16.2f}{cln_mean
- raw_mean:>+10.2f}")
    print(f"{'Медиана ряда, атм':<28}{raw_med:>14.2f}{cln_med:>16.2f}{cln_med -
raw_med:>+10.2f}")
    print(f"{'Стандартное отклонение, атм':<28}{raw_std:>14.2f}{cln_std:>16.2f}{cln_std -
raw_std:>+10.2f}")
    print(f"{'Размах вариации R, атм':<28}{raw_rng:>14.2f}{cln_rng:>16.2f}{cln_rng -
raw_rng:>+10.2f}")


if __name__ == "__main__":
    main()
```

Зачем это делаем: Функция filter_anomalies не удаляет элементы из исходного списка pressures через .pop() или .remove() во время итерации (это классическая ошибка новичков, приводящая к пропуску соседних элементов!), а собирает чистые данные в новый список cleaned.

Шаг 3. Запустить и проверить

```
python main.py
```

Введите свой номер N. Пример работы программы для N = 5:

Пример вывода программы (Вариант А, N = 5):

```
ПОИСК АНОМАЛИЙ И ОЧИСТКА РЯДА ЗАБОЙНОГО ДАВЛЕНИЯ (ВАРИАНТ А)
================================================================
Введите ваш номер по журналу N: 5

Исходный ряд (12 ч):   [170.0, 172.0, 168.0, 134.0, 174.0, 170.0, 166.0, 172.0, 214.0,
168.0, 172.0, 174.0]
Тех. регламент:        [150.00; 190.00] атм
Границы 2.0*sigma:     [136.41; 205.93] атм

ДЕТАЛЬНАЯ ТАБЛИЦА КОНТРОЛЯ ИЗМЕРЕНИЙ
----------------------------------------------------------------
Час      P, атм  Откл. от ср.   Z-оценка   Тех. коридор Статус точки
----------------------------------------------------------------
1        170.00         -1.17       0.07             OK        Норма
2        172.00         +0.83       0.05             OK        Норма
3        168.00         -3.17       0.18             OK        Норма
4        134.00        -37.17       2.14        НАРУШЕН      ВЫБРОС!
5        174.00         +2.83       0.16             OK        Норма
6        170.00         -1.17       0.07             OK        Норма
7        166.00         -5.17       0.30             OK        Норма
8        172.00         +0.83       0.05             OK        Норма
9        214.00        +42.83       2.46        НАРУШЕН      ВЫБРОС!
10       168.00         -3.17       0.18             OK        Норма
```

```
11       172.00         +0.83       0.05             OK        Норма
12       174.00         +2.83       0.16             OK        Норма

РЕЗУЛЬТАТ ФИЛЬТРАЦИИ АНОМАЛИЙ
----------------------------------------------------------------------
Обнаружено выбросов:   2 шт. -> [(4, 134.0), (9, 214.0)]
Очищенный ряд (10 ч):  [170.0, 172.0, 168.0, 174.0, 170.0, 166.0, 172.0, 168.0, 172.0,
174.0]

СРАВНЕНИЕ СТАТИСТИК ДО И ПОСЛЕ ОЧИСТКИ
----------------------------------------------------------------------
Показатель                     До очистки   После очистки Изменение
----------------------------------------------------------------------
Среднее арифметическое, атм         171.17          170.60     -0.57
Медиана ряда, атм                   171.00          171.00     +0.00
Стандартное отклонение, атм          17.38            2.46    -14.92
Размах вариации R, атм               80.00            8.00    -72.00
```

Шаг 4. Ручной контрольный пересчёт (обязательно в отчёт)

Выполните вручную на калькуляторе для своих 12 чисел расчёт среднего, дисперсии, стандартного отклонения σ, границ отсечения [mean - 2σ; mean + 2σ] и Z-оценок для двух аномальных точек (4-й и 9-й час). Запишите расчёт в отчёт.

Образец ручного контрольного пересчёта для отчёта (Вариант А, N = 5):

```
1) Среднее исходного ряда (12 точек):
   mean = 2054.0 / 12 = 171.1667 ≈ 171.17 атм
2) Сумма квадратов отклонений от среднего и СКО (sigma):
   sum((P_i - 171.1667)^2) = 3624.67 -> Дисперсия D = 3624.67 / 12 = 302.0556
   sigma = sqrt(302.0556) = 17.3797 ≈ 17.38 атм
3) Пороги отсечения по правилу 2*sigma:
   Нижний порог:  Low  = 171.1667 - 2 * 17.3797 = 136.41 атм
   Верхний порог: High = 171.1667 + 2 * 17.3797 = 205.93 атм
4) Проверка точек 4-го часа (134.0 атм) и 9-го часа (214.0 атм):
   Час 4: z = |134.0 - 171.17| / 17.38 = 2.14 > 2.0 (и 134.0 < 136.41) -> ВЫБРОС!
   Час 9: z = |214.0 - 171.17| / 17.38 = 2.46 > 2.0 (и 214.0 > 205.93) -> ВЫБРОС!
5) После удаления двух выбросов размах R снизился с 80.00 атм до 174.0 - 166.0 = 8.00 атм,
   а медиана осталась абсолютно неизменной (171.00 атм)!
```

Шаг 5. Закоммитить в Git

```
git init
git add main.py
git commit -m "Вариант А: поиск выбросов давления и фильтрация ряда"
```

Создайте файлы .gitignore и README.md и выполните ещё два коммита:

```
git add .gitignore
git commit -m "Добавить .gitignore"
git add README.md
git commit -m "Добавить документацию README с формулами фильтрации"
```

7. Вариант Б (продвинутый): «Интеллектуальный модуль очистки телеметрии (Z-score vs IQR и импутация)»

7.1. Что мы делаем

Разрабатываем многомодульный пакет в архитектуре src-layout. Программа анализирует 16-часовой ряд забойного давления, содержащий три разнородные аномалии (умеренный сбой, глубокий провал и экстремальную «шпильку»), и сравнивает эффективность двух статистических детекторов: классического метода Z-оценок (k·σ) и робастного метода межквартильного размаха Тьюки (1.5·IQR). Кроме фильтрации (удаления выбросов), программа выполняет импутацию — автоматическое замещение аномальных замеров медианой ряда для сохранения непрерывной часовой сетки.

> **Почему это важно для аналитика:** Когда в ряду есть один гигантский выброс («шпилька»), он сам раздувает стандартное отклонение σ в несколько раз! Из-за этого границы mean ± 2σ становятся слишком широкими, и метод Z-score перестаёт замечать более мелкие сбои (эффект маскирования / masking effect). Метод Тьюки (IQR) опирается на квартили Q1 и Q3, поэтому не обманывается экстремальными выбросами и ловит все три сбоя безошибочно.

7.2. Индивидуальные исходные данные (Вариант Б)

Параметры индивидуального ряда из 16 часов (Вариант Б):

```
N = ваш номер по журналу (например, N = 5)
P0 = 180.0 + 2.5 * N        # базовое давление, атм (при N = 5: P0 = 192.5)
S  = (N % 4) + 1.5          # шаг колебаний, атм (при N = 5: S = 2.5)

# Коэффициенты на 16 часов (часы 1..16):
# Час 5 (коэф. -9.0) — умеренный провал; Час 10 (коэф. +32.0) — гигантская «шпилька»;
# Час 14 (коэф. -11.0) — второй провал давления.
Коэффициенты K (16 шт.):
[0.0, 1.0, -1.0, 2.0, -9.0, 0.0, 1.0, -1.0, 2.0, 32.0, -2.0, 1.0, 0.0, -11.0, 1.0, 2.0]
```

Шаг 1. Создать проект со структурой пакета (src-layout)

```
mkdir anomaly-advanced
cd anomaly-advanced
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте дерево каталогов и файлов:

Структура проекта Варианта Б:

```
anomaly-advanced/
├── .gitignore
├── README.md
└── src/
    └── telemetry_cleaner/
        ├── __init__.py
        ├── config.py
        ├── detectors.py
        └── main.py
```

Шаг 2. Написать код модулей

Файл src/telemetry_cleaner/__init__.py:

```
"""Пакет робастной детекции аномалий и очистки промысловой телеметрии."""

__version__ = "0.5.0"
```

Файл src/telemetry_cleaner/config.py:

```
"""Конфигурационные константы детекторов аномалий."""

Z_K = 2.0                # множитель стандартного отклонения для метода Z-score
IQR_FACTOR = 1.5         # коэффициент Тьюки для межквартильного размаха (1.5 * IQR)

OFFSETS_16H = [
    0.0, 1.0, -1.0, 2.0, -9.0, 0.0, 1.0, -1.0,
    2.0, 32.0, -2.0, 1.0, 0.0, -11.0, 1.0, 2.0,
]
```

Файл src/telemetry_cleaner/detectors.py:

```
"""Математическое ядро: расчёт квартилей, границ Z-score / IQR, фильтрация и импутация."""

from telemetry_cleaner import config


def build_series_16h(student_n: int) -> list[float]:
    """Генерирует 16-часовой ряд забойного давления по номеру студента N."""
    base = 180.0 + 2.5 * student_n
    step = float(student_n % 4) + 1.5
    return [round(base + k * step, 2) for k in config.OFFSETS_16H]


def calc_mean(series: list[float]) -> float:
    """Среднее арифметическое ряда."""
    if not series:
        raise ValueError("Пустой числовой ряд")
    total = 0.0
    for x in series:
        total += x
    return total / len(series)


def calc_median(series: list[float]) -> float:
    """Медиана числового ряда."""
    n = len(series)
    if n == 0:
        raise ValueError("Пустой числовой ряд")
    ordered = sorted(series)
    mid = n // 2
    if n % 2 == 1:
        return ordered[mid]
    return (ordered[mid - 1] + ordered[mid]) / 2.0
```

```
def calc_std(series: list[float], mean_val: float) -> float:
    """Выборочное стандартное отклонение ряда."""
    if not series:
        raise ValueError("Пустой числовой ряд")
    sq_sum = 0.0
    for x in series:
        sq_sum += (x - mean_val) ** 2
    return (sq_sum / len(series)) ** 0.5


def calc_quartiles_iqr(series: list[float]) -> tuple[float, float, float]:
    """Вычисляет первый квартиль Q1, третий квартиль Q3 и межквартильный размах IQR.
    Используется метод Тьюки (медианы младшей и старшей половин отсортированного ряда)."""
    n = len(series)
    if n < 4:
        raise ValueError("Для расчёта квартилей нужно минимум 4 элемента")
    ordered = sorted(series)
    mid = n // 2
    lower_half = ordered[:mid]
    upper_half = ordered[mid:] if n % 2 == 0 else ordered[mid + 1 :]
    q1 = calc_median(lower_half)
    q3 = calc_median(upper_half)
    return q1, q3, round(q3 - q1, 2)


def get_zscore_bounds(series: list[float], k: float = config.Z_K) -> tuple[float, float]:
    """Возвращает границы нормального коридора [mean - k*std; mean + k*std]."""
    m = calc_mean(series)
    s = calc_std(series, m)
    return round(m - k * s, 2), round(m + k * s, 2)


def get_iqr_bounds(series: list[float], factor: float = config.IQR_FACTOR) -> tuple[float,
float]:
    """Возвращает робастные границы Тьюки [Q1 - factor*IQR; Q3 + factor*IQR]."""
    q1, q3, iqr = calc_quartiles_iqr(series)
    return round(q1 - factor * iqr, 2), round(q3 + factor * iqr, 2)


def filter_by_bounds(series: list[float], low: float, high: float) -> list[float]:
    """Возвращает новый список только из нормальных значений в пределах [low; high]."""
    return [x for x in series if low <= x <= high]


def impute_with_median(series: list[float], low: float, high: float) -> list[float]:
    """Возвращает ряд исходной длины, где выбросы заменены на медиану ряда."""
    med = round(calc_median(series), 2)
    return [x if low <= x <= high else med for x in series]
```

Файл src/telemetry_cleaner/main.py:

```
"""Консольный интерфейс сравнения детекторов Z-score и IQR (вариант Б)."""

from telemetry_cleaner import config, detectors
```

```
def main() -> None:
    print("ИНТЕЛЛЕКТУАЛЬНЫЙ МОДУЛЬ ОЧИСТКИ ТЕЛЕМЕТРИИ СКВАЖИНЫ (ВАРИАНТ Б)")
    print("=" * 76)
    raw = input("Введите ваш номер по журналу N: ").strip()
    student_n = int(raw) if raw.isdigit() and int(raw) > 0 else 5

    series = detectors.build_series_16h(student_n)
    mean_v = detectors.calc_mean(series)
    med_v = detectors.calc_median(series)
    std_v = detectors.calc_std(series, mean_v)
    q1, q3, iqr = detectors.calc_quartiles_iqr(series)

    z_low, z_high = detectors.get_zscore_bounds(series, config.Z_K)
    iqr_low, iqr_high = detectors.get_iqr_bounds(series, config.IQR_FACTOR)

    print(f"\nИсходный ряд (16 ч): {series}")
    print("-" * 76)
    print(f"Среднее: {mean_v:.2f} | Медиана: {med_v:.2f} | СКО (sigma): {std_v:.2f} атм")
    print(f"Квартили: Q1 = {q1:.2f} | Q3 = {q3:.2f} | Межквартильный размах IQR =
{iqr:.2f} атм")
    print(f"Коридор Z-score ({config.Z_K}*sigma): [{z_low:.2f}; {z_high:.2f}] атм")
    print(f"Коридор Тьюки ({config.IQR_FACTOR}*IQR):     [{iqr_low:.2f}; {iqr_high:.2f}]
атм")

    cleaned_iqr = detectors.filter_by_bounds(series, iqr_low, iqr_high)
    imputed_iqr = detectors.impute_with_median(series, iqr_low, iqr_high)

    print("\nСРАВНИТЕЛЬНАЯ ТАБЛИЦА ДЕТЕКЦИИ И ИМПУТАЦИИ ПО ЧАСАМ")
    print("-" * 76)
    print(f"{'Час':<5}{'P факт':>10}{'Z-оценка':>10}{'Метод Z-score':>16}{'Метод
1.5*IQR':>16}{'Ряд с заменой':>16}")
    print("-" * 76)

    z_outliers: list[int] = []
    iqr_outliers: list[int] = []

    for idx, val in enumerate(series):
        hour = idx + 1
        z_val = abs(val - mean_v) / std_v if std_v > 0 else 0.0
        is_z_out = val < z_low or val > z_high
        is_iqr_out = val < iqr_low or val > iqr_high
        if is_z_out:
            z_outliers.append(hour)
        if is_iqr_out:
            iqr_outliers.append(hour)

        z_str = "ВЫБРОС" if is_z_out else "Норма"
        iqr_str = "ВЫБРОС!" if is_iqr_out else "Норма"
        imp_val = imputed_iqr[idx]

print(f"{hour:<5}{val:>10.2f}{z_val:>10.2f}{z_str:>16}{iqr_str:>16}{imp_val:>16.2f}")

    cln_mean = detectors.calc_mean(cleaned_iqr)
    cln_std = detectors.calc_std(cleaned_iqr, cln_mean)

    print("-" * 76)
```

```
    print(f"Выбросы по Z-score (часы): {z_outliers} (пропущены умеренные сбои из-за
раздутого СКО!)")
    print(f"Выбросы по 1.5*IQR (часы): {iqr_outliers} (найдены все 3 сбоя датчика)")
    print(f"Очищенный ряд ({len(cleaned_iqr)} точек):  {cleaned_iqr}")
    print(f"После очистки (IQR): Среднее = {cln_mean:.2f} атм, СКО снизилось с {std_v:.2f}
до {cln_std:.2f} атм")


if __name__ == "__main__":
    main()
```

Шаг 3. Запустить и проверить

```
$env:PYTHONPATH="src"
python -m telemetry_cleaner.main
```

Пример вывода программы для N = 5:

Пример вывода программы (Вариант Б, N = 5):

```
ИНТЕЛЛЕКТУАЛЬНЫЙ МОДУЛЬ ОЧИСТКИ ТЕЛЕМЕТРИИ СКВАЖИНЫ (ВАРИАНТ Б)
================================================================================
Введите ваш номер по журналу N: 5

Исходный ряд (16 ч): [192.5, 195.0, 190.0, 197.5, 170.0, 192.5, 195.0, 190.0, 197.5,
272.5, 187.5, 195.0, 192.5, 165.0, 195.0, 197.5]
--------------------------------------------------------------------------------
Среднее: 195.31 | Медиана: 193.75 | СКО (sigma): 21.84 атм
Квартили: Q1 = 190.00 | Q3 = 196.25 | Межквартильный размах IQR = 6.25 атм
Коридор Z-score (2.0*sigma): [151.63; 239.00] атм
Коридор Тьюки (1.5*IQR):     [180.62; 205.62] атм

СРАВНИТЕЛЬНАЯ ТАБЛИЦА ДЕТЕКЦИИ И ИМПУТАЦИИ ПО ЧАСАМ
--------------------------------------------------------------------------------
Час      Р факт  Z-оценка   Метод Z-score   Метод 1.5*IQR   Ряд с заменой
--------------------------------------------------------------------------------
1        192.50      0.13           Норма           Норма          192.50
2        195.00      0.01           Норма           Норма          195.00
3        190.00      0.24           Норма           Норма          190.00
4        197.50      0.10           Норма           Норма          197.50
5        170.00      1.16           Норма         ВЫБРОС!          193.75
6        192.50      0.13           Норма           Норма          192.50
7        195.00      0.01           Норма           Норма          195.00
8        190.00      0.24           Норма           Норма          190.00
9        197.50      0.10           Норма           Норма          197.50
10       272.50      3.53          ВЫБРОС         ВЫБРОС!          193.75
11       187.50      0.36           Норма           Норма          187.50
12       195.00      0.01           Норма           Норма          195.00
13       192.50      0.13           Норма           Норма          192.50
14       165.00      1.39           Норма         ВЫБРОС!          193.75
15       195.00      0.01           Норма           Норма          195.00
16       197.50      0.10           Норма           Норма          197.50
--------------------------------------------------------------------------------
Выбросы по Z-score (часы): [10] (пропущены умеренные сбои из-за раздутого СКО!)
Выбросы по 1.5*IQR (часы): [5, 10, 14] (найдены все 3 сбоя датчика)
```

```
Очищенный ряд (13 точек):  [192.5, 195.0, 190.0, 197.5, 192.5, 195.0, 190.0, 197.5, 187.5,
195.0, 192.5, 195.0, 197.5]
После очистки (IQR): Среднее = 193.65 атм, СКО снизилось с 21.84 до 3.00 атм
```

Шаг 4. Ручной контрольный пересчёт (Вариант Б)

Выполните в отчёте ручной пересчёт для своих 16 чисел: 1) выпишите отсортированный ряд из 16 элементов и разделите его на младшую половину (первые 8 чисел) и старшую половину (последние 8 чисел); 2) найдите медиану младшей половины Q1 и медиану старшей половины Q3; 3) вычислите IQR = Q3 - Q1 и границы [Q1 - 1.5·IQR; Q3 + 1.5·IQR]; 4) объясните, почему метод Z-score (2σ) пропустил выбросы на 5-м и 14-м часах.

Шаг 5. Закоммитить в ветке Git

```
git init
git switch -c feature/anomaly-detection
git add src/telemetry_cleaner/config.py src/telemetry_cleaner/detectors.py
git commit -m "Реализовать детекторы выбросов Z-score и IQR"
git add src/telemetry_cleaner/main.py .gitignore README.md
git commit -m "Добавить таблицу сравнения методов, импутацию и README"
git switch -c main
git merge feature/anomaly-detection
```

8. Обязательные упражнения для обоих вариантов

Все семь упражнений выполняют ВСЕ студенты — и в варианте А, и в варианте Б. Скриншоты и письменные ответы по каждому упражнению обязательно включаются в отчёт.

Упражнение 1. Чувствительность порога Z-оценки (k = 1.5 против k = 3.0)

Последовательно измените в коде пороговый коэффициент Z_THRESHOLD (или Z_K) сначала на жёсткое значение 1.5, а затем на мягкое классическое значение 3.0 («правило трёх сигм»). Запустите скрипт в обоих режимах.

> **Зачем это делаем:** На короткой выборке (12–16 точек) математически невозможно получить Z-оценку сильно выше 3.0 для умеренных сбоев, поэтому правило 3σ на малых выборках часто «слепнет» и пропускает реальные аварии.

Проверка понимания (в отчёт): Приведите таблицу «Коэффициент k | Границы отсечения | Найденные часы выбросов» для k = 1.5, 2.0 и 3.0 и объясните, что такое ложное срабатывание (при малом k) и пропуск сбоя (при большом k).

Упражнение 2. Эффект маскирования выбросов (Masking Effect)

Замените в вашем ряду аномальный всплеск (9-й час в варианте А или 10-й час в варианте Б) на гигантское число 1000.0 атм (короткое замыкание датчика). Посмотрите, как изменилось стандартное отклонение σ и смог ли метод 2σ после этого заметить отрицательный провал давления (4-й час в варианте А или 5-й/14-й час в варианте Б)!

> **Зачем это делаем:** Это главный урок анализа данных: один сверхмощный выброс раздувает

среднее и СКО настолько сильно, что все остальные аномалии оказываются внутри широкого коридора mean ± 2σ и «прячутся» в его тени.

Проверка понимания (в отчёт): Приложите скриншот запуска с числом 1000.0 и письменно объясните, почему провал давления перестал определяться по Z-оценке.

Упражнение 3. Двухпроходная (итеративная) очистка ряда

Доработайте код так, чтобы после первого удаления выбросов программа пересчитывала новое среднее и новое СКО на очищенном ряде и выполняла ВТОРОЙ проход поиска выбросов с обновлёнными границами.

Проверка понимания (в отчёт): Покажите код и скриншот двухпроходной фильтрации. Позволяет ли второй проход обнаружить провал давления из Упражнения № 2, который был замаскирован гигантской «шпилькой» на первом проходе?

Упражнение 4. Опасность удаления элементов из списка во время цикла (.remove / .pop)

Проведите эксперимент: поставьте два выброса подряд (например, сделайте и 4-й, и 5-й час равными 110.0 атм) и попробуйте удалять их прямо внутри цикла for val in pressures: if val < stat_low: pressures.remove(val). Выведите список после такого цикла!

Зачем это делаем: Когда вы удаляете элемент из списка во время итерации по нему, все последующие элементы сдвигаются влево на один индекс, а счётчик цикла шагает дальше — в результате второй подряд идущий выброс перепрыгивается и остаётся в «очищенном» списке!

Проверка понимания (в отчёт): Приложите скриншот ошибки, когда второй соседний выброс остался в списке, и объясните, почему фильтрацию нужно делать через создание нового списка.

Упражнение 5. Импутация (замещение) аномалий вместо удаления

Напишите функцию replace_outliers_with_neighbor_avg(series, low, high), которая не удаляет выброс, а заменяет его полусуммой двух соседних часов: (series[i - 1] + series[i + 1]) / 2 (для крайних точек — на медиану).

Проверка понимания (в отчёт): Приведите код функции, скриншот полученного ряда и ответьте: в каких задачах (например, при расчёте скользящей средней из ЛР № 4 или построении почасового графика) нельзя удалять точки и менять длину ряда?

Упражнение 6. Доля бракованной телеметрии (Процент потерь данных)

Добавьте расчёт показателя качества канала связи: процент отбракованных измерений anomaly_rate = len(anomalies) / len(pressures) * 100 и автоматический вердикт: если брака <= 10% — «Канал исправен», от 10% до 25% — «Требуется калибровка датчика», свыше 25% — «Авария телеметрии».

Проверка понимания (в отчёт): Приведите код доработки, скриншот вывода и ручной расчёт доли брака для вашего ряда.

Упражнение 7. Защита от вырожденного ряда (Константный сигнал / Залипание датчика)

Проверьте работу вашей программы на ряду из одинаковых чисел (например, все 12 часов датчик выдаёт ровно [170.0, 170.0, ..., 170.0] — эффект «залипания» датчика). Чему при этом равно стандартное отклонение σ и межквартильный размах IQR?

Проверка понимания (в отчёт): Покажите скриншот запуска на константном ряде и объясните, почему в формуле Z-оценки abs(val - mean) / std обязательно нужна защита if std > 0 else 0.0.

9. Пример оформления отчёта

- Титульная часть: дисциплина, тема ЛР № 5, ФИО, группа, дата, вариант (А или Б), номер по журналу N;
- Цель работы и версии ПО (Python, Git, VS Code);
- Индивидуальные исходные данные: расчёт базового давления P0, шага S и выписанный массив из 12 (или 16) замеров;
- Ручной контрольный пересчёт: среднее, СКО (или квартили Q1, Q3 и IQR), пороги отсечения и номера часов, попавших в выбросы;
- Ход работы: листинг кода и скриншот основного табличного вывода программы;
- Отчёт по всем 7 обязательным упражнениям (раздел 8) со скриншотами и письменными выводами;
- Письменные ответы на контрольные вопросы (раздел 11) и общее заключение.

Пример начала отчёта (Вариант А):

```
----------------------------------------------------------------------
Отчёт по лабораторной работе № 5
Дисциплина: Программирование и прототипирование ИТ-решений
Студент: Иванов И.И., гр. БПИ-26-01 · Вариант А · N = 5

1. Индивидуальный ряд давления (P0 = 170.0, S = 2.0, 12 часов):
   P = [170.0, 172.0, 168.0, 134.0, 174.0, 170.0, 166.0, 172.0, 214.0, 168.0, 172.0,
174.0]
2. Ручной контрольный пересчёт:
   - Среднее исходного ряда = 2054.0 / 12 = 171.17 атм
   - Стандартное отклонение sigma = sqrt(3624.67 / 12) = 17.38 атм
   - Статистический коридор (2*sigma): [136.41; 205.93] атм
   - Найденные аномалии: час 4 (134.0 атм, z = 2.14) и час 9 (214.0 атм, z = 2.46)
   - После очистки (10 точек): Среднее = 170.60 атм, СКО = 2.46 атм, Размах = 8.00 атм
----------------------------------------------------------------------
```

10. Частые ошибки и их решение

Симптом / Ошибка	Причина и решение
ZeroDivisionError: float division by zero при расчёте Z-оценки	Все элементы ряда одинаковые, поэтому стандартное отклонение std равно 0.0. Добавьте проверку: z = abs(x - mean) / std if std > 0 else 0.0.
Второй из двух подряд идущих выбросов не удаляется из списка	Вы удаляете элементы через series.remove(x) или series.pop(i) прямо внутри цикла for по этому же списку. Создавайте новый список cleaned = []!

Стандартное отклонение получается огромным (сотни атмосфер)	Забыли извлечь квадратный корень из дисперсии (** 0.5) или возвели в квадрат саму сумму вместо отклонений (val - mean) ** 2.
Квартили Q1 и Q3 считаются неверно (Q1 получается больше медианы)	Вы забыли отсортировать ряд через sorted(series) перед делением на младшую и старшую половины.
Метод Z-score (при k = 3) не находит ни одного выброса в ряду из 12 чисел	В коротком ряду сам выброс сильно раздувает СКО. Используйте k = 2.0, двухпроходную очистку или робастный метод 1.5*IQR.
ModuleNotFoundError: No module named 'telemetry_cleaner' (Вариант Б)	Перед запуском пакета в терминале PowerShell не задана переменная окружения: выполните $env:PYTHONPATH="src".

11. Контрольные вопросы для защиты

Ответьте на каждый вопрос письменно в отчёте своими словами, опираясь на написанный вами код:

- **Вопрос 1.** В чём состоит принципиальная разница между поиском аномалий по жёстким технологическим границам [P_min; P_max] и по статистическим критериям (k·σ или 1.5·IQR)? В какой ситуации датчик может выдавать физически допустимое число, которое всё равно является статистическим выбросом?
- **Вопрос 2.** Как вычисляется стандартное отклонение (σ) и Z-оценка точки? Почему при появлении одного экстремального сбоя (например, 1000 атм) метод Z-score перестаёт замечать другие, менее крупные аномалии в том же ряду?
- **Вопрос 3.** Как устроен робастный метод Тьюки на основе межквартильного размаха (1.5·IQR)? Почему квартили Q1 и Q3 и границы отсечения IQR практически не меняются даже при гигантском единичном выбросе?
- **Вопрос 4.** Почему нельзя удалять элементы из списка с помощью .remove() или .pop() непосредственно внутри цикла for x in series:? Продемонстрируйте механизм сдвига индексов на примере двух соседних выбросов.
- **Вопрос 5.** Чем отличаются две стратегии обработки обнаруженных аномалий: фильтрация (удаление точек) и импутация (замещение медианой или средним соседей)? В каких алгоритмах анализа временных рядов удаление точек недопустимо и почему?

ПОЯСНЕНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ № 5

Как пользоваться этим приложением

Основной текст лабораторной работы содержит задание, а данный раздел отвечает на вопрос «почему именно так?». Читайте его параллельно с выполнением: сначала разберите один маленький числовой пример на бумаге, затем перенесите те же действия в Python. Не переписывайте код механически — после каждой функции попробуйте предсказать её результат.

- Если математическая подготовка пока недостаточна, начинайте с разделов «Минимальная математика» и «Разбор одной точки».
- Если Python уже знаком, используйте разделы о выборе метода, маскировании и двухпроходной очистке как материал для углубления.
- На защите важно уметь объяснить не только строку кода, но и смысл величин: что измеряется, в каких единицах и почему точка признана аномальной.

A. Минимальная математика перед началом

Пусть временной ряд давления записан как $x_1, x_2, \ldots, x_n$. Индекс i — это номер измерения (час), а x_i — его значение в атмосферах. В Python список хранит эти значения с нулевой индексацией: первый элемент имеет индекс 0, но для отчёта удобнее показывать «час 1». Поэтому в коде часто используется enumerate(series, start=1).

Обозначение	Простыми словами	Пример для [10, 12, 14]
n	количество измерений	3
Σx_i	сумма всех значений	36
$\bar{x}$	среднее, типичный уровень	36 / 3 = 12
min / max	наименьшее / наибольшее	10 / 14
R	размах: max − min	14 − 10 = 4

1. Среднее арифметическое. Сначала складываем все значения, затем делим сумму на их количество:

$\bar{x} = (x_1 + x_2 + \ldots + x_n) / n = (\Sigma x_i) / n.$

Смысл: если заменить все измерения одним одинаковым числом, среднее будет таким числом, чтобы общая сумма сохранилась. Среднее чувствительно к выбросам: одно значение 1000 резко потянет его вверх.

2. Отклонение и квадрат отклонения. Отклонение $d_i = x_i - \bar{x}$ показывает, насколько точка выше или ниже среднего. Если сложить обычные отклонения, положительные и отрицательные числа взаимно уничтожатся. Поэтому отклонения возводят в квадрат: $d_i^2 = (x_i - \bar{x})^2$.

3. Дисперсия и стандартное отклонение. В этой работе используется деление на n: $D = \Sigma(x_i - \bar{x})^2 / n$, $\sigma = \sqrt{D}$. Это описательная характеристика именно данного набора измерений (её часто называют population standard deviation). В других курсах для оценки генеральной совокупности по выборке применяют n − 1. Нельзя смешивать формулы: если код делит на n, ручной пересчёт тоже должен делить на n.

Единицы важны: дисперсия измеряется в атм², а σ после извлечения корня снова измеряется в атм. Поэтому порог $\bar{x} \pm k\sigma$ можно сравнивать с давлением.

```
mean_val = sum(series) / len(series)
sq_sum = 0.0
for x in series:
    sq_sum += (x - mean_val) ** 2
std = (sq_sum / len(series)) ** 0.5
```

B. Пошаговый числовой пример: почему 134 атм — выброс

Возьмём вариант A для N = 5: P = [170, 172, 168, 134, 174, 170, 166, 172, 214, 168, 172, 174]. Здесь 4-й час — провал, 9-й — всплеск. Среднее уже посчитано в основном тексте: $\bar{x}$ = 171.1667 атм, σ ≈ 17.3797 атм.

Шаг	Расчёт	Результат
1	Отклонение 4-го часа: 134 − 171.1667	−37.1667 атм
2	Модуль отклонения: |−37.1667|	37.1667 атм
3	Z = |x − x̄| / σ	37.1667 / 17.3797 ≈ 2.14
4	Сравнение с k = 2.0	2.14 > 2.0 → статистически подозрительно
5	Порог снизу: x̄ − 2σ	171.1667 − 34.7594 ≈ 136.41
6	Проверка коридором	134 < 136.41 → выброс

Для 9-го часа аналогично: |214 − 171.1667| / 17.3797 ≈ 2.46, а 214 > 205.93. Значит, срабатывают оба признака: точка выходит и за технологический коридор, и за статистический порог. В отчёте полезно явно написать: «аномалия — это не просто необычное число; это число, нарушающее выбранное правило обнаружения».

C. Как читать программу сверху вниз

Любую реализацию удобно разбирать как конвейер из пяти этапов. Каждый этап получает данные, выполняет одну понятную операцию и передаёт результат дальше.

Этап	Вход	Что происходит	Выход
1. Генерация	N студента	вычисляются P0, S и коэффициенты K	исходный список pressures
2. Описательная статистика	pressures	среднее, медиана, σ, min, max	числа для порогов и таблицы
3. Детекция	точка + пороги	проверки low <= x <= high и/или z > k	список (час, значение) аномалий
4. Обработка	исходный ряд + аномалии	фильтрация или замещение	cleaned / imputed
5. Контроль	исходный и новый ряды	сравнение статистик и проверка результата	таблица и выводы

Важно различать «обнаружить» и «исправить». Детектор только отвечает «подозрительно / нормально». Фильтр удаляет подозрительные элементы. Импутация сохраняет длину ряда, заменяя значение. Ошибка в одном из этих понятий приводит к неправильному выводу, даже если программа запускается без исключения.

D. Три разных правила и почему они не взаимозаменяемы

1) Технологический коридор [P_min; P_max] задаётся предметной областью. Он не зависит от того, насколько «разбросан» текущий ряд. Если датчик физически не может выдавать 214 атм, значение запрещено даже при малом числе наблюдений.

2) Метод k·σ строит коридор вокруг среднего: [$\bar{x}$ − kσ; $\bar{x}$ + kσ]. Он удобен для приблизительно симметричных данных, но среднее и σ сами изменяются под влиянием выброса. Поэтому экстремальная «шпилька» может спрятать остальные ошибки.

3) Метод Тьюки использует медиану центральных половин. После сортировки ряда нижняя половина даёт Q1, верхняя — Q3; IQR = Q3 − Q1; границы: Low = Q1 − 1.5·IQR, High = Q3 + 1.5·IQR. Этот метод робастен: единичное очень большое значение обычно не меняет центральные 50% ряда.

Правило	Сильная сторона	Ограничение	Когда выбрать
Технологические границы	учитывает физику процесса	нужны достоверные нормативы	контроль допустимого оборудования
k·σ	простая формула и понятный z-score	чувствителен к выбросам	примерно симметричный стабильный сигнал
1.5·IQR	устойчив к экстремальным значениям	на очень коротком ряду может быть грубым	сравнение и робастная очистка

E. Квартили без «магии»

Алгоритм для чётного ряда из 16 элементов: (1) сделать копию и отсортировать; (2) первые 8 чисел образуют lower_half, последние 8 — upper_half; (3) Q1 — среднее 4-го и 5-го элементов lower_half; Q3 — среднее 4-го и 5-го элементов upper_half. Например, для [1, 2, 3, 4, 5, 6, 7, 8] медиана равна (4 + 5) / 2 = 4.5. Для нижней половины [1,2,3,4] Q1 = (2 + 3) / 2 = 2.5; для верхней [5,6,7,8] Q3 = (6 + 7) / 2 = 6.5; IQR = 4.0.

Почему нужна sorted(series), а не сортировка самого списка? sorted создаёт новый список и оставляет исходные измерения в хронологическом порядке. Если изменить исходный ряд, часы перестанут соответствовать значениям, и таблица станет неверной.

```
ordered = sorted(series)       # исходный series не меняется
mid = len(ordered) // 2
lower = ordered[:mid]
upper = ordered[mid:]
q1 = calc_median(lower)
q3 = calc_median(upper)
iqr = q3 - q1
```

F. Безопасные конструкции Python: индексы, пустые ряды, нулевой разброс

- Проверяйте пустой список до деления на len(series). Иначе получится ZeroDivisionError или неинформативный результат.
- Для z-score проверяйте std > 0. В константном ряду x = $\bar{x}$ для каждой точки, но формула формально делит 0 на 0. Практическое решение: z = 0.0 if std == 0 else abs(x - mean) / std.
- Не удаляйте элементы из списка, по которому сейчас идёт for. После remove/pop индексы сдвигаются, и соседний выброс может быть пропущен.
- Для границ используйте цепное сравнение low <= x <= high: оно означает одновременно x не меньше low и x не больше high.

```
def filter_by_bounds(series, low, high):
    cleaned = []
    for x in series:
        if low <= x <= high:
            cleaned.append(x)
    return cleaned

def impute_with_median(series, low, high, median):
    return [x if low <= x <= high else median for x in series]
```

Преимущество нового списка cleaned: исходные данные остаются доказательством того, что именно пришло от датчика. В реальном проекте исходный файл нельзя перезаписывать без резервной копии.

G. Маскирование и двухпроходная очистка — наглядное объяснение

Пусть в ряду есть обычные значения около 190, провал 170 и гигантская шпилька 1000. Шпилька увеличивает сумму квадратов $(1000 - \bar{x})^2$ во много раз, поэтому σ становится огромным. Коридор $\bar{x} \pm 2\sigma$ расширяется, и 170 может оказаться внутри него. Это не означает, что 170 стало нормальным: изменился сам измерительный масштаб.

Двухпроходный алгоритм:

- Проход 1: вычислить среднее и σ исходного ряда, отметить очевидные выбросы.
- Удалить только найденные точки или временно исключить их из расчёта.
- Проход 2: пересчитать $\bar{x}$ и σ по оставшимся точкам и снова проверить исходный ряд.
- Сравнить результат с IQR: если методы расходятся, не выбирать «лучший» автоматически, а объяснить причину и предметный смысл.

Двухпроходность — эвристика, а не универсальное доказательство. В отчёте укажите, какое правило применялось на каждом проходе, иначе невозможно воспроизвести результат.

H. Как оформить ручной контрольный пересчёт

Ручной пересчёт нужен не для дублирования всей программы, а для проверки ключевых мест. Для своего N заполните шаблон:

Что записать	Формула / действие	Ваш результат
Параметры	P0 = …; S = …	
Сумма и количество	ΣP = …; n = …	
Среднее	$\bar{x} = \Sigma P / n$	
Сумма квадратов	$\Sigma(P_i - \bar{x})^2 = \ldots$	
СКО	$\sigma = \sqrt{(\Sigma / n)}$	
Порог	Low = $\bar{x} - k\sigma$; High = $\bar{x} + k\sigma$	
Проверка выброса	$z_i = \lvert P_i - \bar{x} \rvert / \sigma$	
Итог	часы … признаны аномальными	

Правило самопроверки: округляйте только при выводе. В промежуточных вычислениях храните полную точность, иначе граница может сдвинуться на сотые и точка окажется по другую сторону порога. Если ручной и программный результаты отличаются на 0.01–0.02 из-за округления, покажите промежуточные значения и объясните это.

I. Два варианта отчёта: что должно быть доказано

Раздел отчёта	Минимальное доказательство	Частая недоработка
Исходные данные	P0, S, N и полный ряд	указан только N, ряд не выписан
Математика	формулы и один подробный расчёт	есть только скриншот запуска
Код	листинг функций с пояснением входов/выходов	код вставлен без комментариев
Детекция	таблица по каждому часу и список аномалий	показано только количество
Очистка	исходный, filtered и/или imputed ряд	неясно, удаляли или заменяли
Контроль	ручной пересчёт и сравнение ДО/ПОСЛЕ	округлённые числа без формул
Упражнения	ответ на «почему», а не только скриншот	скриншот без вывода
Git	не менее трёх осмысленных коммитов	коммиты сделаны одной датой без содержания

J. Контрольный список перед сдачей

- Я могу объяснить разницу между технологическим и статистическим порогом.
- Я знаю, почему в z-score используется abs и что делать при std = 0.
- Я могу вручную получить среднее, σ, границы и проверить хотя бы две точки.
- Я понимаю, почему IQR рассчитывается после сортировки копии ряда.
- Я не изменяю исходный список во время обхода.
- Я различаю фильтрацию (длина меняется) и импутацию (длина сохраняется).
- В таблице сохранены номер часа, исходное значение, z-score и статус каждого измерения.
- Для каждого из семи упражнений есть эксперимент, результат и письменное объяснение.
- В README есть формулы, команда запуска и пример вывода.
- В Git есть минимум три коммита, названия которых описывают сделанную работу.

K. Вопросы для самопроверки с подсказками

Вопрос: Почему нельзя объявить любое далёкое от среднего значение ошибкой датчика?
Подсказка: Потому что редкое значение может быть реальным режимом процесса; нужны физический контекст и выбранное правило.

Вопрос: Что произойдёт со средним и σ после добавления 1000 атм?
Подсказка: Оба обычно вырастут; σ особенно сильно, так как отклонение возводится в квадрат.

Вопрос: Почему медиана часто стабильнее среднего?
Подсказка: Она определяется порядковым положением и не так сильно меняется от одного крайнего значения.

Вопрос: Почему сортировка не должна менять хронологию?
Подсказка: Сортировка нужна только для Q1/Q3; исходный порядок нужен для номера часа и анализа временной динамики.

Вопрос: Когда удаление точек недопустимо?
Подсказка: Когда важна равномерная временная сетка: график, скользящее среднее, прогнозирование и расчёт производных.

L. Итоговая формулировка заключения (шаблон)

«В лабораторной работе я сформировал(а) индивидуальный временной ряд по номеру N = …, вычислил(а) среднее … атм и стандартное отклонение … атм по формуле с делением на n. По правилу … были обнаружены часы … . После фильтрации длина ряда изменилась с … до …, а после импутации осталась равной … . Сравнение Z-score и IQR показало … . На экспериментах с k, гигантским выбросом, соседними выбросами и константным рядом я убедился(лась), что … .»

Главная идея работы: алгоритм поиска аномалий — это не одна формула, а последовательность: корректно представить данные → вычислить характеристики → сформулировать критерий → проверить каждую точку → выбрать безопасный способ обработки → доказать результат ручным и программным контролем.